

Functional languages

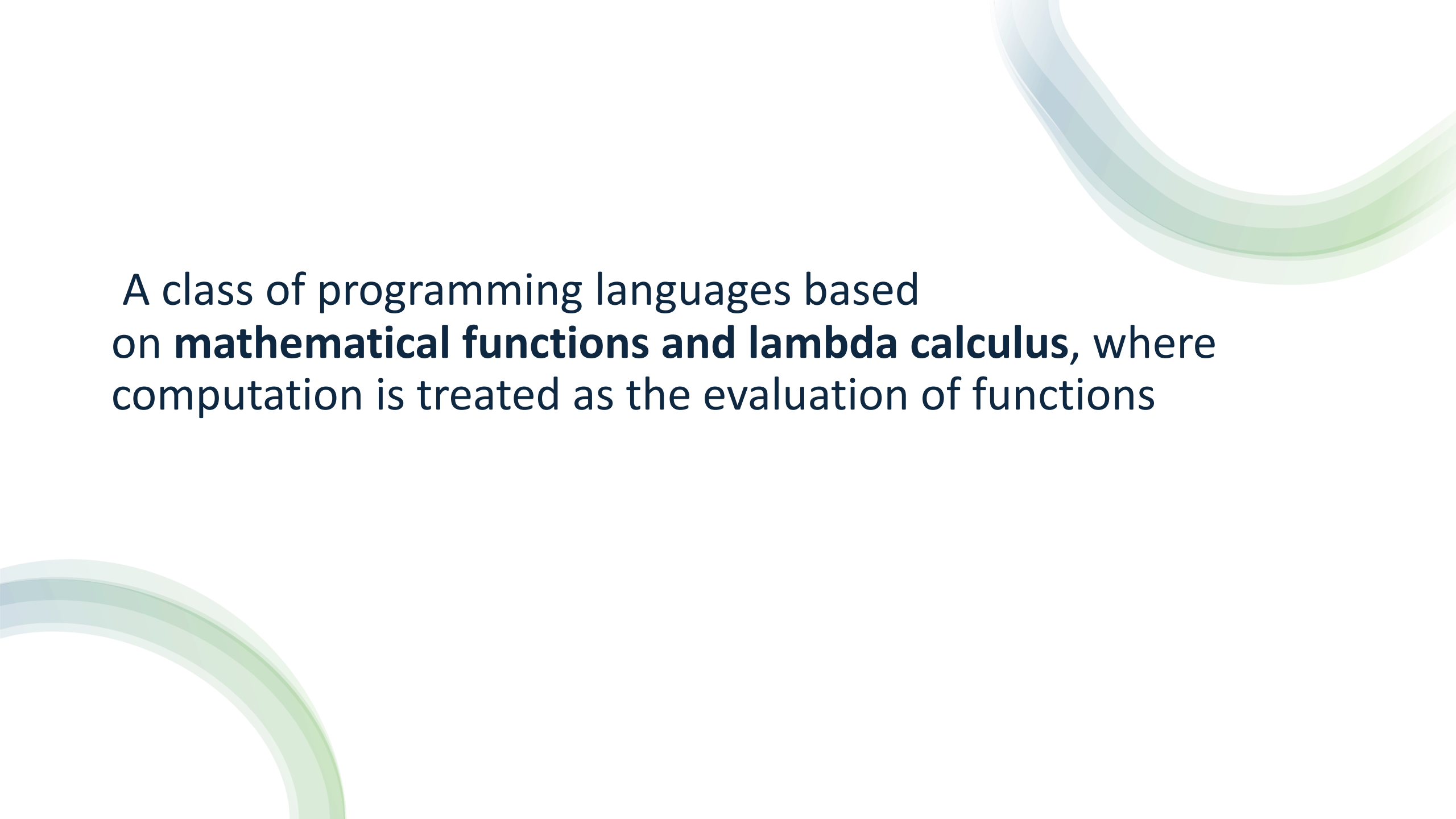
Higher order functions, Referential transparency, Lambda Calculus, Strict and lazy Evaluation, Functional Programming

lec7

BAKD

DoCE

2026

The image features a white background with decorative curved lines in the top right and bottom left corners. These lines are composed of multiple overlapping layers in shades of light blue and green, creating a sense of depth and movement.

A class of programming languages based
on **mathematical functions and lambda calculus**, where
computation is treated as the evaluation of functions

What Is Functional Language?

- Functional programming languages focus on using functions for computations.
- Functions in these languages act like mathematical equations, producing consistent outputs for given inputs.
- "First-class functions" allow functions to be treated like other data types, enabling them to be passed, returned, and stored.

What Is Functional Language?

- Recursion replaces traditional loops, with functions calling themselves until a base case is reached.
- Immutability ensures that once a variable is set, its value cannot be changed, reducing unexpected bugs.
- List processing capabilities enable easy manipulation and analysis of data lists.

- Functional programming reduces errors by maintaining consistent outputs and avoiding state changes.
- Support for parallel programming allows multiple computations to occur simultaneously, enhancing efficiency.

- Lazy evaluation means expressions are only computed when needed, optimizing performance.
- Examples of functional programming languages include Haskell, Lisp, OCaml, and aspects of JavaScript.
- Each language implements functional principles in unique ways while sharing core concepts.
- Functional programming promotes clean, efficient, and maintainable code through the use of functions.

Immutability



Pure function



No side effects



Referential transparency



Functional programming is built around several key principles:

- **Pure Functions:**
- **Immutability:**
- **First-Class and Higher-Order Functions:**
- **Recursion:**
- **Lazy Evaluation (optional):**

Core Concepts

- **Pure Functions:** A pure function's output depends only on its inputs and has no side effects (e.g., modifying external variables, I/O operations). Calling a pure function with the same arguments will always produce the same result, making code more predictable and easier to test and debug (a property called referential transparency).
- **Immutability:** Data, once created, cannot be changed. Instead of modifying a data structure, you create a new one with the desired updates, preserving the original data. This eliminates many concurrency issues in parallel programming.

- **First-Class and Higher-Order Functions:** Functions are treated as ordinary values that can be passed as arguments to other functions, returned as results, and assigned to variables. Higher-order functions, such as map, filter, and reduce, are commonly used to abstract over common programming patterns and work with collections of data.
- **Recursion:** In the absence of traditional loops (like for or while statements) that rely on mutable states, iteration is typically achieved through recursion, where a function calls itself until a base condition is met.
- **Lazy Evaluation (optional):** Some functional languages (like Haskell) use lazy evaluation, which means expressions are not evaluated until their results are actually needed. This can improve performance and allow for the creation of infinite lists or data streams

Advantages

- **Fewer Bugs:** The restriction on side effects and mutable state makes programs less prone to bugs and easier to reason about.
- **Easier Testing:** Pure functions are straightforward to unit test because their behavior is entirely determined by their inputs.
- **Concurrency and Parallelism:** Code with no shared mutable state is inherently thread-safe, simplifying the development of concurrent and parallel applications.
- **Modularity:** Functional programming encourages breaking problems into small, composable functions, which leads to more modular and reusable code.

Examples of Functional Languages

Some languages are purely functional, while many others support functional programming features:

- Pure Functional Languages: Haskell, Clean, Miranda.
- Impure/Multi-Paradigm Languages: Common Lisp, Scheme, Clojure, Erlang, OCaml, Scala, F#. Mainstream languages like Python, JavaScript, Java (since Java 8), and C# have also incorporated functional features like first-class functions and lambdas.

<https://youtu.be/-ik11z0etFM>

Key characteristics of functional language

Using a functional programming language offers several advantages, particularly for certain types of applications and development practices:

- Functions are treated like any other data type, allowing them to be passed as arguments to other functions and returned as values.
- Data is generally treated as immutable, meaning it cannot be modified after creation. This helps prevent side effects and makes code easier to reason about.

Cont..

- Recursion is a fundamental technique in FP for solving problems by breaking them down into smaller, similar subproblems.
- Functions can take other functions as arguments or return functions as results, enabling powerful abstractions.
- Expressions are evaluated only when their results are needed, potentially improving performance in specific scenarios.

Why use a functional language?

They offer several advantages:

1. Conciseness and readability — Functional code can often be more concise and easier to understand due to its declarative nature.
2. Reliability — The absence of mutable state reduces the likelihood of bugs and makes code more predictable.
3. Parallelism — Functional languages are well-suited for parallel programming, as functions can be executed independently without shared state.

4. Testability — The declarative style, focusing on what should be done rather than how, and lack of side effects make testing easier.
5. Efficiency — Lazy evaluation can optimize performance, especially in certain scenarios.
6. Big data handling — Functional languages are widely used in big data frameworks (e.g., Apache Spark, Akka) because they manage concurrent tasks and scale efficiently.

What are some examples of functional programming languages?

- Besides Haskell, as we mentioned above, there are other popular examples of functional programming languages. Here are they:
- Scala — designed for the Java Virtual Machine (JVM), offering a blend of object-oriented and functional programming features.
- Clojure — A dynamically typed language known for its interactive development environment (REPL) and focus on functional programming.
- Elixir — A language excelling in building concurrent and scalable applications, known for its focus on [functional programming principles](#).
- F# — A statically typed functional language for the .NET platform, praised for its strong type system and integration with other .NET languages.
- Elm — A functional language for building user interfaces, known for its type system and predictable behavior.
- OCaml — A general-purpose programming language that combines functional and object-oriented programming paradigms.
- Lisp — One of the oldest languages, known for its flexibility and use in AI research. Lisp supports multiple paradigms but is heavily functional.

- Functional programming emphasizes immutability, first-class functions, and declarative code, making it well-suited for applications requiring reliability and parallelism.
- Key benefits include improved readability, easier parallelism due to the absence of shared state, and increased predictability from reduced side effects.
- Functional programming is particularly well-suited for applications involving symbolic computation, concurrency, and large-scale data processing.
- Popular functional languages like Haskell, Scala, and Elixir offer strong type systems and powerful abstractions.

Higher-Order Functions (HOFs)

Definition

A **higher-order function** is a function that:

- Takes one or more **functions as parameters**, and/or
- **Returns a function** as its result

Why Higher-Order Functions are Important

- Enable **functional programming** style
- Improve **code reuse and abstraction**
- Support **composition and modular design**
- Simplify complex operations on data

Used heavily in:

- Functional languages (Haskell, ML, Lisp)
- Modern languages (Python, JavaScript, Scala)

Basic Properties

A language supports higher-order functions if it allows:

1. Functions to be treated as **first-class values**
2. Functions to be:
 - Assigned to variables
 - Passed as arguments
 - Returned from other functions
 - Stored in data structures

Examples in Different Languages

Python Example

python

```
def apply(f, x):  
    return f(x)  
  
def square(n):  
    return n * n  
  
result = apply(square, 5)    # result = 25
```

javascript

```
function apply(f, x) {  
    return f(x);  
}  
  
function square(n) {  
    return n * n;  
}  
  
console.log(apply(square, 5));
```

haskell

```
apply f x = f x  
square n = n * n
```


Closures and Higher-Order Functions

- A **closure** is a function that remembers variables from its defining scope
- Often created when returning functions

Used for:

- Encapsulation
- State preservation

Advantages

- High level of abstraction
- Less repetitive code
- Easier parallelization
- Expressive and concise programs

Disadvantages

- Harder to understand for beginners
- Debugging is more complex
- Performance overhead (function calls, lambdas)

Term

First-class function

Higher-order function

Lambda expression

Closure

Function composition

Callback

Meaning

Functions treated like values

Function that takes/returns functions

Anonymous function

Function with remembered environment

Combining functions

Function passed as argument

1. What is a higher-order function?

A function that takes another function as an argument or returns a function as its result.

2. What is meant by first-class functions?

Functions that can be assigned to variables, passed as arguments, and returned from other functions.

3. Give two examples of higher-order functions.

map, filter, reduce.

4. What is a callback function?

A function passed as an argument to another function to be executed later.

5. What is a closure?

A function that retains access to variables from its defining lexical scope even after the outer function has finished.

6. Why are higher-order functions useful?

They improve abstraction, reduce code duplication, and enable functional programming patterns.

7. Which languages strongly support higher-order functions?

Haskell, Lisp, ML, Python, JavaScript, Scala.

8. What is function composition?

Combining two or more functions so that the output of one becomes the input of another.

9. Distinguish between higher-order function and ordinary function.

- Ordinary: Takes data and returns data
- Higher-order: Takes functions or returns functions

10. What is the relation between closures and higher-order functions?

Closures are often created when higher-order functions return inner functions that capture variables.

Referential transparency,

Definition

- **An expression is said to be referentially transparent if it can be replaced by its value without changing the behavior of the program.**

In other words:

- **The same expression always produces the same result and has no side effects.**

Key Characteristics of referential T

A referentially transparent expression:

- Always gives the **same output for the same input**
- Does **not modify any external state**
- Does **not depend on mutable variables, I/O, or time**
- Has **no side effects**

Simple Example

python

```
def add(a, b):  
    return a + b
```

Expression:

SCSS

```
add(2, 3)
```

- Always evaluates to 5
- Can be replaced by 5 anywhere

✓ Referentially transparent

Non-Referentially Transparent Example

```
python
```

```
x = 10
```

```
def f(a):  
    return a + x
```

- If `x` changes, result changes
- Depends on external state

✗ Not referentially transparent

Another Non-Transparent Example (Side Effect)

python

```
def g():  
    global x  
    x = x + 1  
    return x
```

- Modifies global variable
- Calling twice gives different results

✗ Not referentially transparent

Importance of Referential Transparency

1. Easier Reasoning

- Program behavior predictable
- Mathematical reasoning possible

2. Easier Debugging & Testing

- No hidden state
- No unexpected interactions

3. Enables Compiler Optimizations

- Common subexpression elimination
- Memoization
- Lazy evaluation

4. Safe Parallel Execution

- No shared state
- No race conditions

Referential Transparency and Functional Programming

Strongly supported in:

- Haskell
- ML
- Pure functional subsets

Partially supported in:

- Python
- Java (with immutable objects)

Not supported naturally in:

- C (due to pointers, globals)
- C++ (mutable state)

Relation to Pure Functions

A **pure function**:

- Always returns same result for same input
- Has no side effects

Therefore:

- All pure functions are referentially transparent.

But:

- Impure functions are not referentially transparent.

Common Causes of Loss of Referential Transparency

Cause

Global variables

Mutable state

I/O operations

Random numbers

Time functions

Exceptions

Example

Using/modifying globals

Changing shared data

Printing, reading input

`rand()`

`time()`

When they affect control flow unpredictably

Examples in Programming Languages

Referentially Transparent (Haskell)

h

```
square x = x * x
```

Always same result → transparent

Not Transparent (C)

c

```
int x = 5;  
int f(int a) {  
    return a + x;  
}
```

Depends on global `x`

Advantages

- Predictable behavior
- Formal verification easier
- Safe concurrency
- Better optimization

Disadvantages / Limitations

- Hard to model:
 - Input/output
 - State changes
 - Interactive systems
- Often requires:
 - Monads (Haskell)
 - Immutable data structures

Q&A

1. What is referential transparency?

An expression is referentially transparent if it can be replaced by its value without changing program behavior.

2. What is the key property of referential transparency?

Same input always produces same output and no side effects occur.

3. Give an example of a referentially transparent function.

$$f(x) = x * x$$

4. Give one example where referential transparency is violated.

A function that modifies a global variable or reads current time.

5. Why is referential transparency important?

It simplifies reasoning, debugging, optimization, and parallel execution.

6. What is the relation between pure functions and referential transparency?

Pure functions are referentially transparent because they have no side effects.

7. Do I/O operations preserve referential transparency? Why?

No, because input and output depend on external state and environment.

8. Which programming paradigm emphasizes referential transparency?

Functional programming.

9. Can imperative languages support referential transparency?

Partially, by avoiding mutable state and side effects.

10. How does referential transparency help compiler optimization?

It allows replacing expressions with values, enabling memoization and common subexpression elimination.